

Orientação a Objetos

Modificadores de acesso e atributos de classe

14/5/2025



Professor

Me. Jonas Pontes

 @jn.pontes

0 saldo da Conta



Não é possível sacar um valor superior
saldo + limite

```
public boolean sacar(double valor) {  
    if (this.saldo + this.limite >= valor) {  
        this.saldo -= valor;  
        return true;  
    }  
    return false;  
}
```



0 saldo da Conta



Manipulando apenas o método sacar:

- 0 limite não poderá ser negativo;
- 0 saldo nunca será menor que -limite.

E se alguém fizer:

- `conta1.saldo = -14000;`
- `conta1.limite = -1000;`

É possível



0 saldo da Conta



Uma solução seria sempre verificar se o valor atribuído ao saldo satisfaz a condição que ele não seja menor que a negação do limite.

- Mas isso é repetição de código;
- E alguém pode simplesmente esquecer disso.



A solução!



Modificadores de acesso ou de visibilidade

- São palavras-chave usadas para controlar a visibilidade de classes, atributos e métodos;
- Determinam quem pode acessar o quê dentro do código;
- Estabelecem níveis de **encapsulamento**;



Modificadores



Modificador	Acesso permitido a	Palavra-chave
Público	Qualquer classe, de qualquer pacote	public
Protegido	Classes do mesmo pacote ou subclasses	protected
Sem modificador	Somente classes do mesmo pacote	
Privado	Somente dentro da mesma classe	private



Modificadores



public

- Pode ser acessado de qualquer lugar do projeto (inclusive fora do pacote);
- Usado para métodos e atributos que devem estar disponíveis amplamente;
 - Costuma ser o padrão para métodos;
 - Pouco recomendado para atributos.



Modificadores



private

- Pode ser acessado somente dentro da própria classe.
- É a base do encapsulamento: usamos para proteger atributos e obrigar o uso de métodos acessores e modificadores.
 - É o padrão que deve ser usado em atributos.



Modificadores



protected

- Acesso permitido dentro do mesmo pacote ou em subclasse;

default (sem palavra-chave)

- Acesso package-private, ou seja, acessível apenas dentro do mesmo pacote.



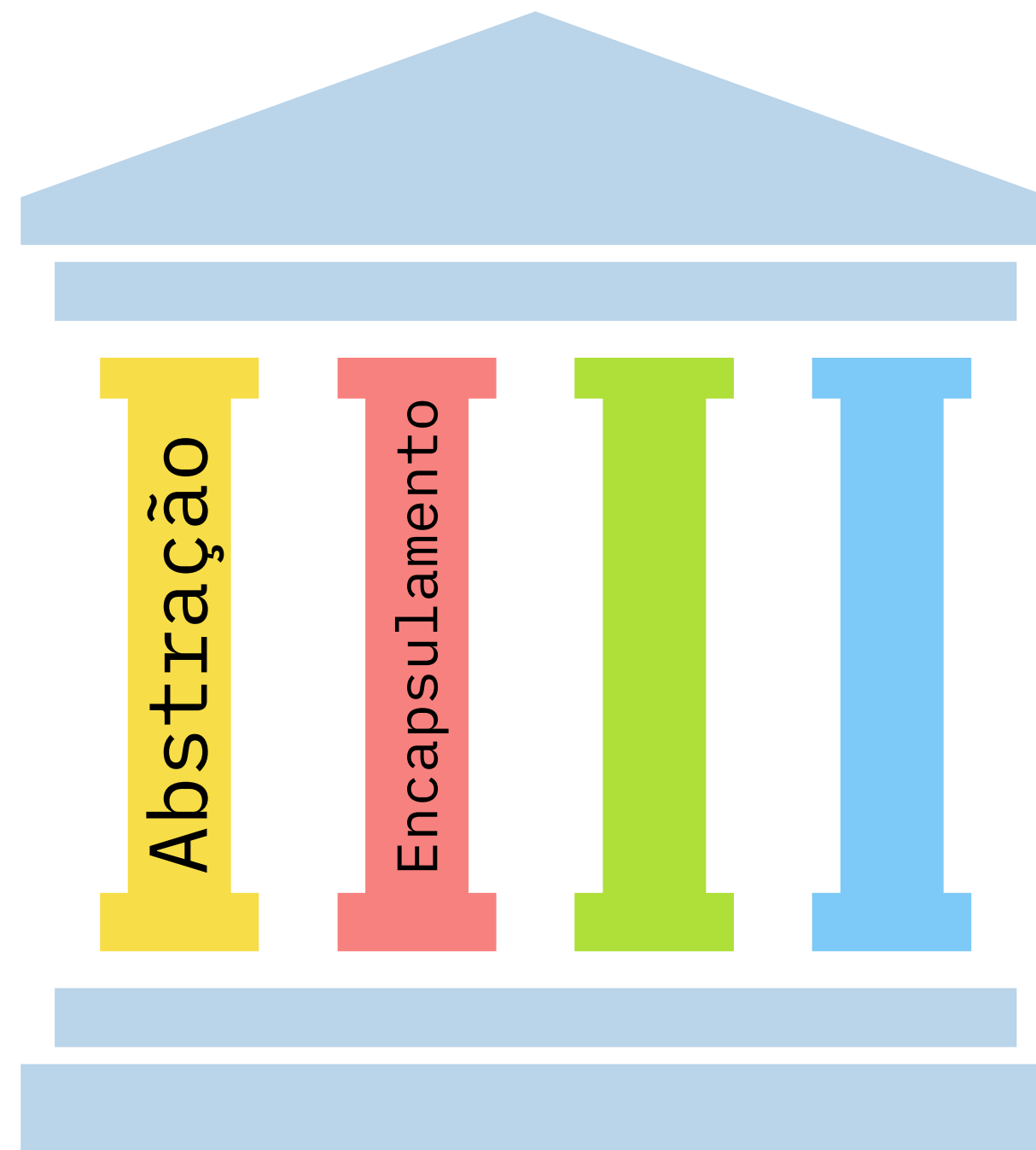
Boas práticas



- Sempre usar **private** para atributos e fornecer acesso controlado com getters e setters.
- Evitar **public** em atributos, pois quebra o encapsulamento.
- Usar **protected** somente quando há herança planejada.
- Usar **default** (sem modificador) quando você quer restringir o uso apenas ao pacote, útil em projetos maiores.



Encapsulamento



Encapsulamento



É o princípio de esconder os detalhes internos de uma classe e expor apenas o necessário.

Ele permite:

- Proteger os dados;
- Controlar o acesso a atributos e métodos;

Facilitar a manutenção do código;

Reforçar a coesão e a modularidade.



Encapsulamento



Como é feito:

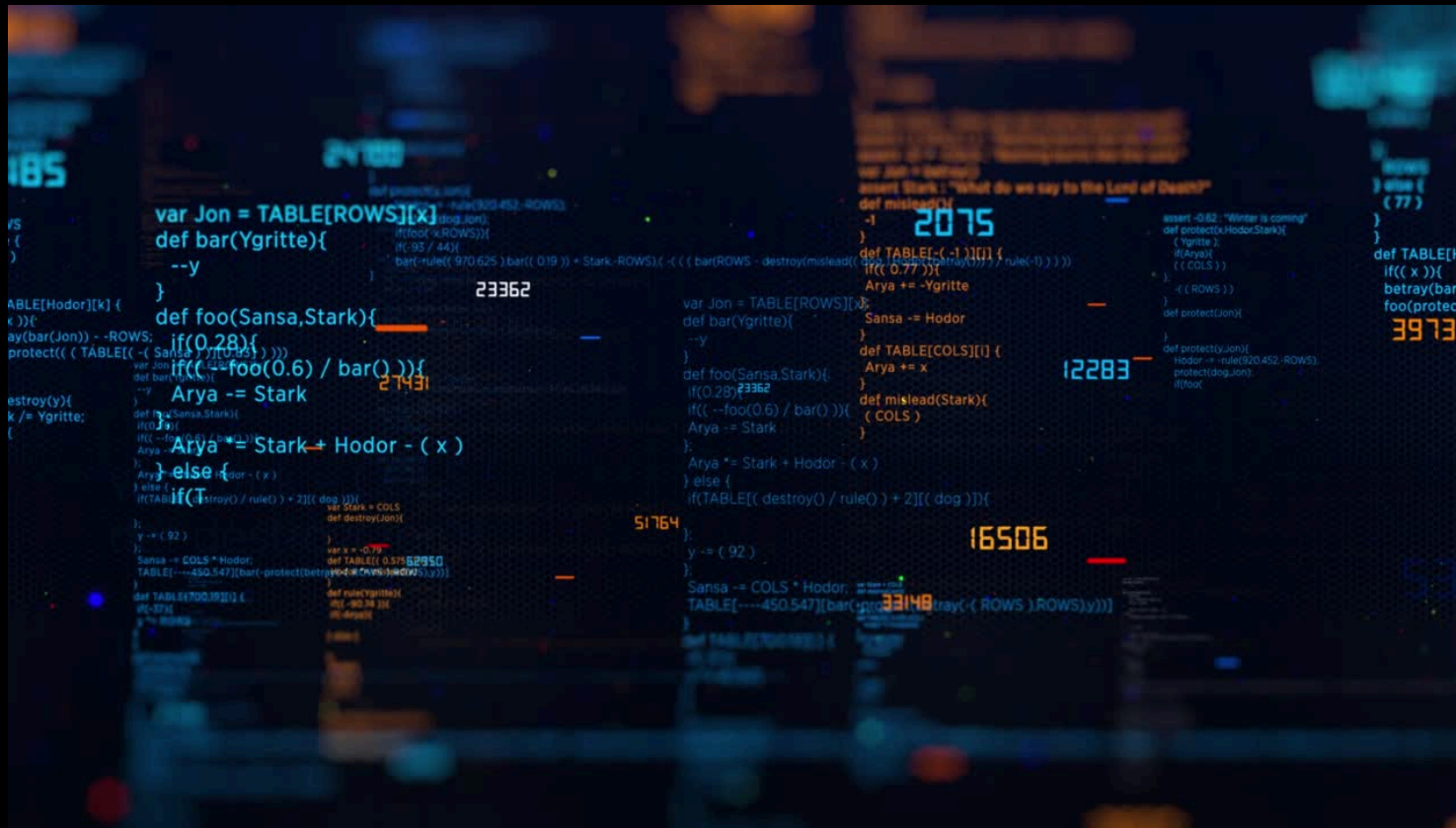
- `private`: protege os atributos;
- `public`: fornece métodos de acesso controlado (getters e setters).

Construtores: definição dos estados iniciais dos objetos.





Vamos à prática

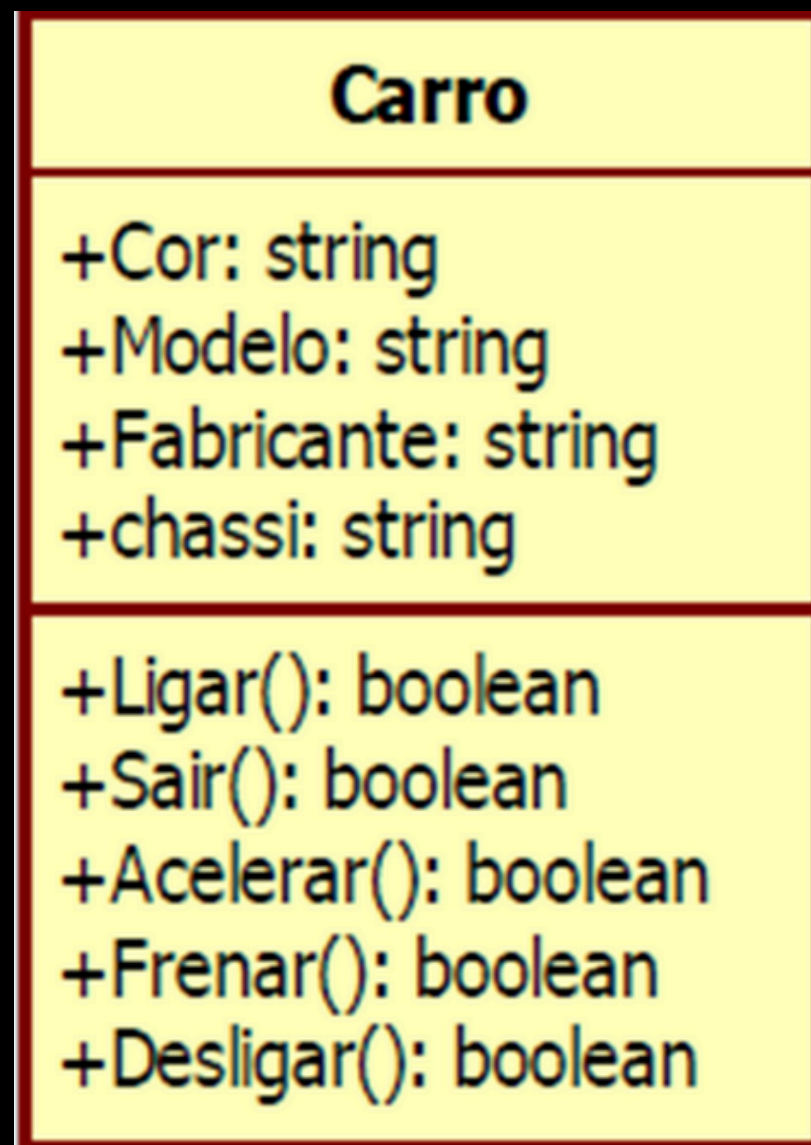




Objetos:

- Em UML a representação de um objeto se parece com a de classes, mas apresentam valores reais para cada atributo;
 - Tem seus nomes sublinhados e possuem o nome da classe logo a seguir, separados por um sinal de dois pontos (:).





Pratique



Crie uma porta. Para isso, responda as questões:

- Quais são os atributos necessário para a criação dessa classe?
- Quais são os métodos necessários para a criação dessa classe?
- Modele, em UML, essa classe.

